



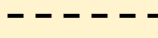
Memoria Caché



Arquitectura de Computadoras



Actividad 4



Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Horario: Mar y Jue de 9:00 a 11:00 A.M.

Profesor: Jose Juan Meza Espinoza

Fecha: 04/10/2025

Preguntas de memoria caché

❖ ¿Qué es la memoria caché y cuál es su principal objetivo dentro de la jerarquía de memoria de un sistema computacional?

La memoria caché es una memoria pequeña, rápida y costosa situada entre la CPU y la memoria principal (RAM).

Su principal objetivo es **reducir el tiempo promedio de acceso a los datos** que necesita el procesador, almacenando temporalmente las copias de los datos e instrucciones más usados.

Dentro de la **jerarquía de memoria**, actúa como un intermediario que acelera la comunicación entre la CPU (muy rápida) y la RAM (más lenta).

❖ Explica la diferencia entre localidad temporal y localidad espacial en el contexto de la memoria caché.

- **Localidad temporal:** Si un dato o instrucción se usa una vez, es probable que se vuelva a usar pronto.
Ejemplo: una variable usada dentro de un bucle.
→ La caché guarda esos datos recientes para accesos futuros rápidos.
- **Localidad espacial:** Si se accede a una dirección de memoria, es probable que pronto se acceda a direcciones cercanas.
Ejemplo: recorrer un arreglo secuencialmente.
→ Por eso, la caché carga bloques completos en lugar de solo una palabra.

❖ Define los conceptos de acierto de caché (hit) y fallo de caché (miss). ¿Por qué son relevantes para el rendimiento del sistema?

- **Acierto (hit):** Ocurre cuando el dato solicitado ya está en la caché.
→ El acceso es muy rápido.
- **Fallo (miss):** Ocurre cuando el dato no está en la caché y debe traerse desde la memoria principal.
→ Es más lento.

Una alta tasa de aciertos mejora significativamente el rendimiento del sistema, ya que reduce los accesos lentos a la RAM.

- ❖ ¿Qué diferencias existen entre una correspondencia directa, asociativa y asociativa por conjuntos en el diseño de caché? Da un ejemplo sencillo de cada una.

Diferencias entre correspondencia directa, asociativa y asociativa por conjuntos		
Tipo de correspondencia	Descripción	Ejemplo
Directa	Cada bloque de memoria tiene una única posición posible en la caché.	Si la memoria tiene 8 bloques y la caché 4, el bloque 5 solo puede ir a la posición 1.
Asociativa	Cualquier bloque puede colocarse en cualquier línea de la caché.	Mayor flexibilidad pero más complejidad en búsqueda.
Asociativa por conjuntos	La caché se divide en conjuntos, y cada bloque puede ir a cualquiera de las líneas del conjunto correspondiente.	Por ejemplo, una caché de 4 conjuntos con 2 líneas cada uno (2-way).

- ❖ ¿Qué son los algoritmos de reemplazo en caché? Menciona y explica brevemente al menos dos (ejemplo: FIFO, LRU).

Cuando ocurre un fallo y la caché está llena, se debe reemplazar un bloque existente. Los algoritmos más comunes son:

- **FIFO (First In, First Out):**
Se reemplaza el bloque que lleva más tiempo en la caché. Fácil de implementar, pero no siempre eficiente.
- **LRU (Least Recently Used):**
Se reemplaza el bloque que menos recientemente se usó. Mejora el rendimiento aprovechando la localidad temporal, aunque requiere más hardware o registros.

- ❖ ¿Cuál es la diferencia entre las políticas de escritura write-through y write-back? ¿En qué casos conviene usar una u otra?

Diferencia entre write-through y write-back			
Política	Descripción	Ventajas	Desventajas
Write-through	Toda escritura se hace tanto en la caché como en la memoria principal.	Mantiene la memoria coherente.	Más tráfico a memoria, más lenta.
Write-back	La escritura se hace solo en la caché y se actualiza la memoria cuando el bloque es reemplazado.	Menos tráfico, más rápido.	Riesgo de incoherencia si hay varios procesadores.

Cuando usar:

- Write-through: sistemas donde la coherencia es crítica (por ejemplo, multiprocesadores).
- Write-back: cuando se busca máximo rendimiento en sistemas monoprocesador.

❖ Explica los tipos de fallos de caché (forzosos, de capacidad y de conflicto) y da un ejemplo de cada uno.

- **Fallos forzosos (compulsory):**
Ocurren la primera vez que se accede a un dato.
Ejemplo: al iniciar un programa.
Son inevitables.
- **Fallos de capacidad:**
Suceden cuando la caché no puede contener todos los datos necesarios para una tarea.
Ejemplo: recorrer un arreglo que excede el tamaño total de la caché.
- **Fallos de conflicto:**
En cachés de correspondencia directa o por conjuntos, cuando dos bloques compiten por la misma posición.
Ejemplo: dos direcciones que se asignan al mismo índice de caché.

❖ ¿Cómo influye el tamaño del bloque de caché en la tasa de fallos? Menciona ventajas y desventajas de bloques grandes.

- **Bloques grandes:**
 - **Ventajas:** aprovechan mejor la localidad espacial.
 - **Desventajas:** aumentan la latencia de carga y la probabilidad de fallos de capacidad (menos bloques totales).
- **Bloques pequeños:**
 - **Ventajas:** permiten almacenar más bloques distintos.
 - **Desventajas:** menos aprovechamiento de la localidad espacial.

❖ ¿Qué ventajas ofrece utilizar varios niveles de caché (L1, L2, L3) en una arquitectura moderna?

- **L1:** más pequeña y rápida, integrada en el núcleo del CPU.
- **L2:** más grande pero más lenta, aún cercana al procesador.
- **L3:** compartida entre núcleos, optimiza la comunicación.

Ventajas:

- Menor latencia promedio de acceso.
- Mejor aprovechamiento del espacio de caché.
- Escalabilidad en procesadores multinúcleo.

❖ ¿Qué papel juegan los compiladores en la optimización del uso de la memoria caché? Menciona alguna técnica (ejemplo: fusión de bucles o arrays).

Los compiladores pueden reorganizar el código para **maximizar la localidad**.
Ejemplo de técnica:

- **Fusión de bucles (loop fusion):** combina varios bucles que recorren los mismos datos, reduciendo accesos dispersos.
- **Bloqueo de bucles (loop blocking o tiling):** divide grandes matrices en bloques pequeños que caben en caché, reduciendo fallos.

Así, el compilador **reduce los fallos de caché** y mejora el rendimiento sin cambiar el hardware.

Conclusión

La memoria caché es un elemento clave para equilibrar la diferencia de velocidad entre la CPU y la RAM. Su efectividad depende de: la localidad de los programas, el diseño de la caché (asociatividad, tamaño de bloque, políticas de reemplazo y escritura) y las estrategias de software que optimicen su uso. En conjunto, estos factores determinan el rendimiento global del sistema, haciendo que las operaciones se realicen de forma más rápida y eficiente.

Bibliografía

- ❖ UCAM Universidad Católica de Murcia. (2015, 5 mayo). Estructura de computadores - Unidad 4.1 Jerarquía de memoria - José Luis Abellán Miguel [Video]. YouTube.
<https://www.youtube.com/watch?v=HwYrfXIwock>
- ❖ UCAM Universidad Católica de Murcia. (2015a, mayo 5). Estructura de computadores - 4.2 Memoria de caché - José Luis Abellán Miguel [Video]. YouTube.
<https://www.youtube.com/watch?v=AeJSk6Q9Tvg>